T3000 API & Protocol

Troubleshooting Guide · built Nov 13, 2026

Troubleshooting Guide

Quick reference for common issues and solutions.

Quick Checklist

System Not Responding

1. Check API server running (port 9103)
2. Verify database accessible
3. Check browser console (F12)
4. Restart API server
5. Clear browser cache

Data Not Updating

1. Verify device connection (green status)
2. Check auto-refresh enabled
3. Click manual refresh
4. Review communication logs
5. Check network connectivity

Slow Performance

1. Reduce displayed points
2. Increase refresh interval
3. Close unused browser tabs
4. Check system resources
5. Archive old data

Error Messages

Database Locked

Solution: Wait for retry (up to 51s), reduce concurrent operations

Connection Timeout

Solution: Increase timeout, check network, verify device online

JSON Parse Error

Solution: API endpoint missing or returning HTML error page

Getting Help

See Device Troubleshooting for detailed solutions.

Advanced Troubleshooting

Diagnostic Tools

Health Check Endpoint

TYPESCRIPT

```typescript
interface HealthCheckResponse {
  status: 'healthy' | 'degraded' | 'unhealthy';
  uptime: number;
  database: {
    connected: boolean;
    responseTime: number;
  };
  devices: {
    total: number;
    online: number;
    offline: number;
  };
  memory: {
    used: number;
    total: number;
  };
}

// Check system health
const health = await fetch('/api/health');
const status: HealthCheckResponse = await health.json();

if (status.database.responseTime > 1000) {
  console.warn('Database slow, may need optimization');
}
```

Log Analysis

TYPESCRIPT

```typescript
// Fetch recent errors from API
const logs = await fetch('/api/logs?level=error&limit=100');
const errors = await logs.json();

// Analyze error patterns
const errorCounts = errors.reduce((acc, log) => {
  const key = log.message.split(':')[0];
  acc[key] = (acc[key] || 0) + 1;
  return acc;
}, {});

console.table(errorCounts);
```

Network Diagnostics

TYPESCRIPT

```typescript
class NetworkDiagnostics {
  async ping(host: string, port: number): Promise<boolean> {
    try {
      const response = await fetch(
        `http://${host}:${port}/api/ping`,
        { signal: AbortSignal.timeout(5000) }
      );
      return response.ok;
    } catch {
      return false;
    }
  }

  async traceroute(deviceId: number): Promise<DiagnosticResult> {
    const start = performance.now();

    try {
      const response = await fetch(`/api/devices/${deviceId}/ping`);
      const latency = performance.now() - start;

      return {
        success: response.ok,
        latency,
        hops: []  // Would require backend support
      };
    } catch (error) {
      return {
        success: false,
        error: error.message,
        latency: performance.now() - start
      };
    }
  }

  async checkBandwidth(): Promise<number> {
    const start = performance.now();
    const testSize = 1024 * 1024;  // 1 MB
```

```
    const response = await fetch('/api/bandwidth-test');
    await response.arrayBuffer();

    const duration = (performance.now() - start) / 1000;
    return (testSize / duration) / 1024;  // KB/s
  }
}
```

Common Issues and Fixes

Database Lock Debugging

TYPESCRIPT

```typescript
// Monitor database lock events
class DatabaseMonitor {
  private lockEvents: LockEvent[] = [];

  logLock(operation: string, duration: number) {
    this.lockEvents.push({
      operation,
      duration,
      timestamp: Date.now()
    });

    // Alert on long locks
    if (duration > 5000) {
      console.error(`Long database lock: ${operation} took ${duration}ms`);
    }
  }

  getStatistics() {
    const avg = this.lockEvents.reduce((sum, e) => sum + e.duration, 0)
                / this.lockEvents.length;
    const max = Math.max(...this.lockEvents.map(e => e.duration));

    return { avg, max, count: this.lockEvents.length };
  }
}

// Use wrapper to track locks
async function withLockMonitoring<T>(
  operation: string,
  fn: () => Promise<T>
): Promise<T> {
  const start = Date.now();
  try {
    return await fn();
  } finally {
    dbMonitor.logLock(operation, Date.now() - start);
```

```
  }
}
```

Memory Leak Detection

TYPESCRIPT

```typescript
class MemoryMonitor {
  private samples: MemorySample[] = [];

  startMonitoring(intervalMs = 10000) {
    setInterval(() => {
      if (performance.memory) {
        this.samples.push({
          timestamp: Date.now(),
          used: performance.memory.usedJSHeapSize,
          total: performance.memory.totalJSHeapSize,
          limit: performance.memory.jsHeapSizeLimit
        });

        // Keep last 100 samples
        if (this.samples.length > 100) {
          this.samples.shift();
        }

        this.checkForLeaks();
      }
    }, intervalMs);
  }

  private checkForLeaks() {
    if (this.samples.length < 10) return;

    const recent = this.samples.slice(-10);
    const older = this.samples.slice(-20, -10);

    const recentAvg = recent.reduce((sum, s) => sum + s.used, 0) / recent.length;
    const olderAvg = older.reduce((sum, s) => sum + s.used, 0) / older.length;

    const growthRate = (recentAvg - olderAvg) / olderAvg;

    if (growthRate > 0.1) {  // 10% growth
      console.warn(`Possible memory leak detected: ${(growthRate * 100).toFixed(1)}% grc
    }
```

```
    }
}
```

WebSocket Connection Issues

TYPESCRIPT

```typescript
class WebSocketDiagnostics {
  private reconnectAttempts = 0;
  private connectionErrors: Error[] = [];

  diagnose(ws: WebSocket): DiagnosticReport {
    const report: DiagnosticReport = {
      state: ws.readyState,
      reconnectAttempts: this.reconnectAttempts,
      recentErrors: this.connectionErrors.slice(-5),
      recommendations: []
    };

    if (this.reconnectAttempts > 5) {
      report.recommendations.push(
        'Excessive reconnection attempts. Check server availability.'
      );
    }

    const errorTypes = this.connectionErrors.map(e => e.message);
    if (errorTypes.filter(m => m.includes('timeout')).length > 3) {
      report.recommendations.push(
        'Frequent timeouts detected. Check network latency.'
      );
    }

    return report;
  }

  logError(error: Error) {
    this.connectionErrors.push(error);
    if (this.connectionErrors.length > 50) {
      this.connectionErrors.shift();
    }
  }
}
```

Performance Profiling

API Request Profiler

TYPESCRIPT

```typescript
class APIProfiler {
  private requests = new Map<string, RequestMetrics>();

  async profile<T>(
    endpoint: string,
    request: () => Promise<T>
  ): Promise<T> {
    const start = performance.now();

    try {
      const result = await request();
      this.recordSuccess(endpoint, performance.now() - start);
      return result;
    } catch (error) {
      this.recordFailure(endpoint, performance.now() - start);
      throw error;
    }
  }

  private recordSuccess(endpoint: string, duration: number) {
    const metrics = this.getMetrics(endpoint);
    metrics.totalRequests++;
    metrics.successfulRequests++;
    metrics.totalDuration += duration;
    metrics.avgDuration = metrics.totalDuration / metrics.successfulRequests;

    if (duration > metrics.maxDuration) {
      metrics.maxDuration = duration;
    }
    if (duration < metrics.minDuration) {
      metrics.minDuration = duration;
    }
  }

  private recordFailure(endpoint: string, duration: number) {
    const metrics = this.getMetrics(endpoint);
```

```
    metrics.totalRequests++;
    metrics.failedRequests++;
  }

  getReport(): ProfileReport {
    const report: ProfileReport = [];

    for (const [endpoint, metrics] of this.requests) {
      report.push({
        endpoint,
        ...metrics,
        successRate: metrics.successfulRequests / metrics.totalRequests
      });
    }

    return report.sort((a, b) => b.avgDuration - a.avgDuration);
  }
}

// Usage
const profiler = new APIProfiler();

const data = await profiler.profile(
  '/api/devices/389001',
  () => fetch('/api/devices/389001').then(r => r.json())
);

console.table(profiler.getReport());
```

Component Render Profiler

TYPESCRIPT

```typescript
import { onMounted, onUpdated } from 'vue';

export function useRenderProfiler(componentName: string) {
  let renderCount = 0;
  let lastRender = Date.now();

  onMounted(() => {
    console.log(`[${componentName}] Mounted`);
  });

  onUpdated(() => {
    renderCount++;
    const now = Date.now();
    const timeSinceLastRender = now - lastRender;

    console.log(
      `[${componentName}] Render #${renderCount} ` +
      `(${timeSinceLastRender}ms since last)`
    );

    if (timeSinceLastRender < 16) {  // Less than 1 frame (60fps)
      console.warn(
        `[${componentName}] Rendering too frequently! ` +
        `Consider memoization or debouncing.`
      );
    }

    lastRender = now;
  });
}
```

Automated Recovery

Circuit Breaker Pattern

TYPESCRIPT

```typescript
enum CircuitState {
  CLOSED,
  OPEN,
  HALF_OPEN
}

class CircuitBreaker {
  private state = CircuitState.CLOSED;
  private failureCount = 0;
  private lastFailureTime = 0;
  private readonly threshold = 5;
  private readonly timeout = 60000;  // 1 minute

  async execute<T>(operation: () => Promise<T>): Promise<T> {
    if (this.state === CircuitState.OPEN) {
      if (Date.now() - this.lastFailureTime > this.timeout) {
        this.state = CircuitState.HALF_OPEN;
      } else {
        throw new Error('Circuit breaker is OPEN');
      }
    }

    try {
      const result = await operation();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  private onSuccess() {
    this.failureCount = 0;
    this.state = CircuitState.CLOSED;
  }
```

```
  private onFailure() {
    this.failureCount++;
    this.lastFailureTime = Date.now();

    if (this.failureCount >= this.threshold) {
      this.state = CircuitState.OPEN;
      console.error('Circuit breaker tripped to OPEN state');
    }
  }
}
```

Self-Healing

TYPESCRIPT

```typescript
class SelfHealingSystem {
  private healthChecks: HealthCheck[] = [];

  registerHealthCheck(check: HealthCheck) {
    this.healthChecks.push(check);
  }

  async monitorAndHeal(intervalMs = 30000) {
    setInterval(async () => {
      for (const check of this.healthChecks) {
        const healthy = await check.check();

        if (!healthy) {
          console.warn(`Health check failed: ${check.name}`);

          if (check.autoHeal) {
            try {
              await check.heal();
              console.log(`Auto-healed: ${check.name}`);
            } catch (error) {
              console.error(`Failed to heal ${check.name}:`, error);
            }
          }
        }
      }
    }, intervalMs);
  }
}

// Example health checks
const healingSystem = new SelfHealingSystem();

healingSystem.registerHealthCheck({
  name: 'Database Connection',
  async check() {
    try {
      await fetch('/api/health/db');
      return true;
```

```
    } catch {
      return false;
    }
  },
  async heal() {
    // Restart database connection
    await fetch('/api/admin/restart-db', { method: 'POST' });
  },
  autoHeal: true
});

healingSystem.registerHealthCheck({
  name: 'Memory Usage',
  async check() {
    if (performance.memory) {
      const usage = performance.memory.usedJSHeapSize /
                    performance.memory.jsHeapSizeLimit;
      return usage < 0.9;  // Less than 90%
    }
    return true;
  },
  async heal() {
    // Clear caches
    window.location.reload();
  },
  autoHeal: false  // Require manual intervention
});
```

Next Steps

- Performance Tuning
- Best Practices
- FAQ